

# AI Inference Optimization

Detailed Overview of My AI Inference Engine Optimization Internship at SoyNet

Moonsu Kim

# From open-source models to optimized inference engines

I rebuilt inference support around real-time service constraints: throughput, GPU memory, compatibility, and verification.

## SCOPE

# 7 models

Vision and NLP model families

## CORE WORK

# CUDA + CFG

GPU preprocessing, postprocessing,  
layer config

## VALIDATED RESULT

# 2.3x FPS

YOLOX-X benchmark improvement

## MEMORY

# 800MB

Reduced from 1800MB on YOLOX-X

## What I did

Debugged PyTorch input/output pipelines and model execution paths

Implemented preprocessing and postprocessing modules in C++/CUDA

Translated operation graphs into engine-specific layer config files

## Why it matters

Lower latency for real-time AI services

Lower GPU memory footprint and serving cost

Accuracy-preserving migration from research code to engine runtime

# End-to-end optimization process

I handled the flow from open-source model analysis to engine-ready implementation and benchmark reporting.

01

## Model sourcing

Download and inspect open-source model code

02

## Input pipeline

Debug preprocessing and tensor layout

03

## GPU modules

Implement C++/CUDA preprocessing and custom layers

04

## Graph tracing

Build DAG-style model architecture maps

05

## Engine config

Write layer-level .cfg files and verify outputs

06

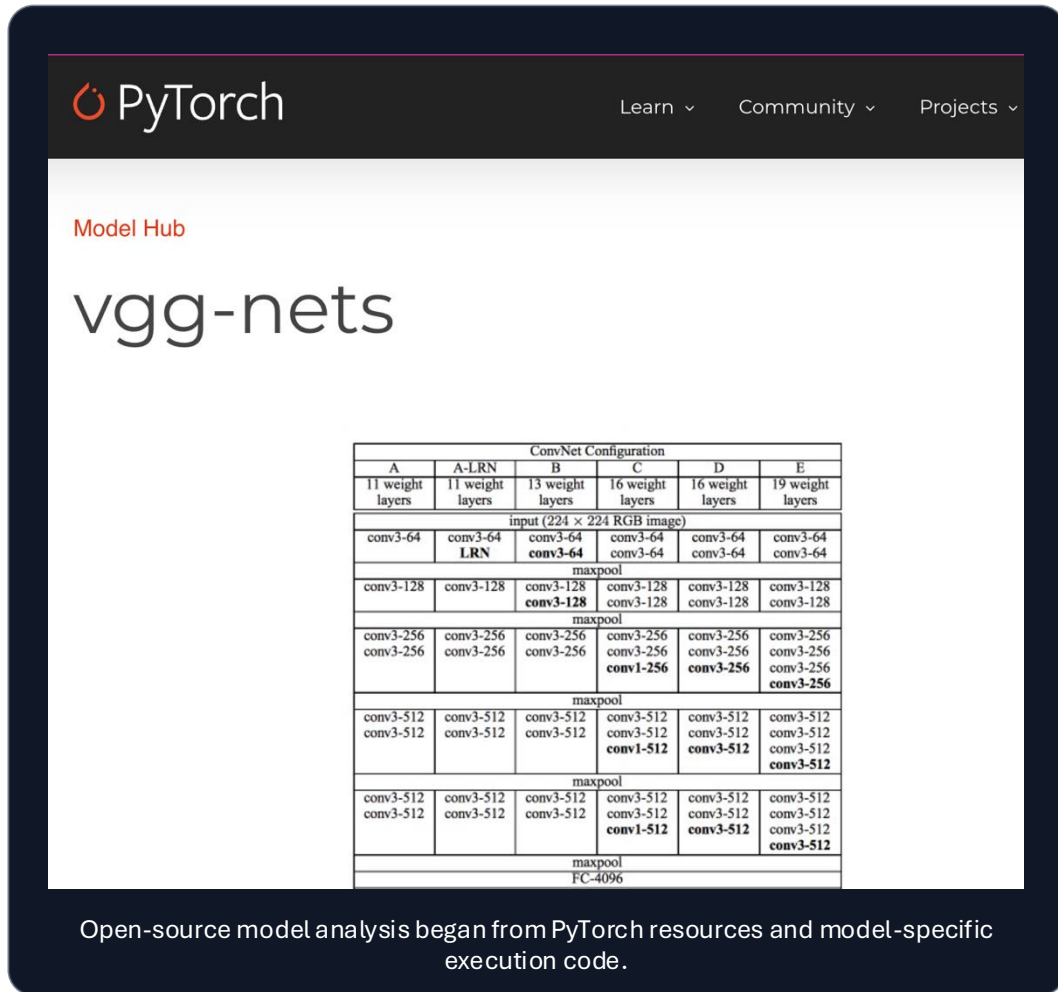
## Benchmark

Measure FPS and GPU memory after optimization

**Key principle: validate compatibility early at each layer, so optimization does not accumulate numerical or structural mismatch.**

# Multiple model families

The project covered classification, segmentation, object detection, pose estimation, and NLP models.



PyTorch

Learn ▾ Community ▾ Projects ▾

Model Hub

## vgg-nets

| ConvNet Configuration       |                        |                        |                                     |                                     |                                     |
|-----------------------------|------------------------|------------------------|-------------------------------------|-------------------------------------|-------------------------------------|
| A                           | A-LRN                  | B                      | C                                   | D                                   | E                                   |
| 11 weight layers            | 11 weight layers       | 13 weight layers       | 16 weight layers                    | 16 weight layers                    | 19 weight layers                    |
| input (224 × 224 RGB image) |                        |                        |                                     |                                     |                                     |
| conv3-64                    | conv3-64<br>LRN        | conv3-64<br>conv3-64   | conv3-64<br>conv3-64                | conv3-64<br>conv3-64                | conv3-64<br>conv3-64                |
| maxpool                     |                        |                        |                                     |                                     |                                     |
| conv3-128                   | conv3-128              | conv3-128<br>conv3-128 | conv3-128<br>conv3-128              | conv3-128<br>conv3-128              | conv3-128<br>conv3-128              |
| maxpool                     |                        |                        |                                     |                                     |                                     |
| conv3-256<br>conv3-256      | conv3-256<br>conv3-256 | conv3-256<br>conv3-256 | conv3-256<br>conv3-256<br>conv1-256 | conv3-256<br>conv3-256<br>conv3-256 | conv3-256<br>conv3-256<br>conv3-256 |
| maxpool                     |                        |                        |                                     |                                     |                                     |
| conv3-512<br>conv3-512      | conv3-512<br>conv3-512 | conv3-512<br>conv3-512 | conv3-512<br>conv3-512<br>conv1-512 | conv3-512<br>conv3-512<br>conv3-512 | conv3-512<br>conv3-512<br>conv3-512 |
| maxpool                     |                        |                        |                                     |                                     |                                     |
| conv3-512<br>conv3-512      | conv3-512<br>conv3-512 | conv3-512<br>conv3-512 | conv3-512<br>conv3-512<br>conv1-512 | conv3-512<br>conv3-512<br>conv3-512 | conv3-512<br>conv3-512<br>conv3-512 |
| maxpool                     |                        |                        |                                     |                                     |                                     |
| FC-4096                     |                        |                        |                                     |                                     |                                     |

Open-source model analysis began from PyTorch resources and model-specific execution code.

## CLASSIFICATION

VGG16 / VGG19  
EfficientNet-B6

## SEGMENTATION

FCN

## OBJECT DETECTION

YOLOX-X

## POSE ESTIMATION

PoseNet Transpose-h-a4

## LANGUAGE

BART Paragraph

# Moved input preparation into GPU-friendly modules

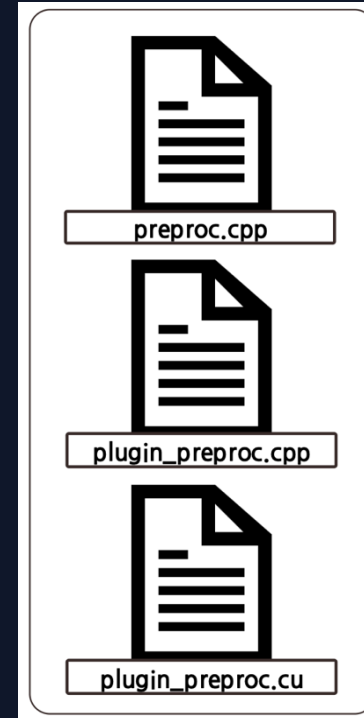
I debugged each model input pipeline, then rebuilt the recurring operations in C++/CUDA modules.

```
input_image = Image.open(filename) input_image: <PIL.JpegImagePlugin.JpegImageFile>
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
input_tensor = preprocess(input_image)
input_batch = input_tensor.unsqueeze(0) # create a mini-batch as expected by the model

# move the input and model to GPU for speed if available
if torch.cuda.is_available():
    input_batch = input_batch.to('cuda')
    model.to('cuda')

with torch.no_grad():
```

PyTorch-side transforms became explicit native modules.



preproc.cpp  
plugin\_preproc.cpp  
plugin\_preproc.cu

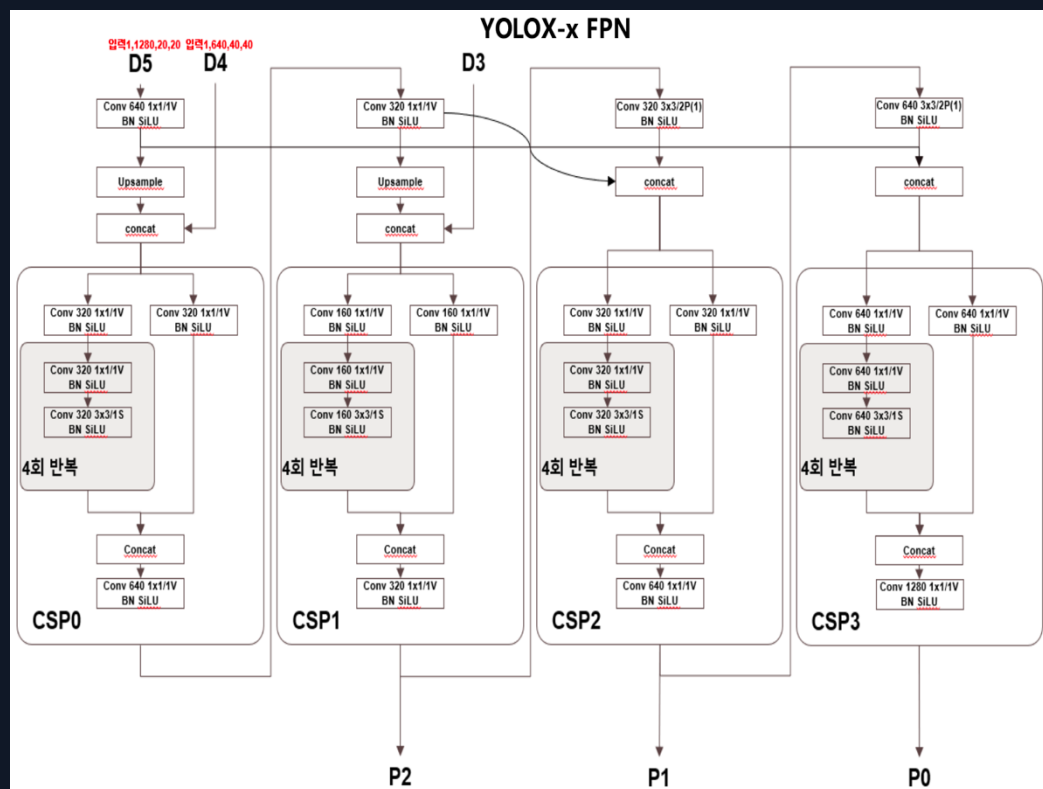
## Implemented operations

Resize  
BGR to RGB  
Crop  
Normalization  
Padding

Result: less Python-side overhead  
and a cleaner path into the  
optimized engine runtime.

# Turned model execution into an explicit layer-level specification

Analyzed source code to reconstruct model DAGs, then converted them into engine-compatible config files.



YOLOX-X FPN DAG: operation paths, layer characteristics, and tensor flow.



```
bot-sort-yolox-x.cfg  X bot-sort-baseline.cfg  bot-sort.cpp  nms.cpp
172 [conv2d] input=P2 channel=320 refname=STEM0
173 [conv2d] input=P1 channel=320 refname=STEM1
174 [conv2d] input=P0 channel=320 refname=STEM2
175 ## CLS conv ##
176 [conv2d] input=STEM0 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
177 [conv2d] input=STEM1 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
178 [conv2d] input=STEM2 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
179 ## REG conv ##
180 [conv2d] input=STEM0 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
181 [conv2d] input=STEM1 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
182 [conv2d] input=STEM2 channel=320 kernel=3 pad_tf=same [conv2d] channel=320 kernel=3 pad_t
183
184 ## CLS PREDS ##
185 [conv2d] input=CLS_0 channel=$CLASS_COUNT weight_order=wa act=sigmoid refname=CLSP_0
186 [conv2d] input=CLS_1 channel=$CLASS_COUNT weight_order=wa act=sigmoid refname=CLSP_1
187 [conv2d] input=CLS_2 channel=$CLASS_COUNT weight_order=wa act=sigmoid refname=CLSP_2
188 ## REG PREDS ##
189 [conv2d] input=REG_0 channel=4 weight_order=wa act=identity refname=REGP_0
190 [conv2d] input=REG_1 channel=4 weight_order=wa act=identity refname=REGP_1
191 [conv2d] input=REG_2 channel=4 weight_order=wa act=identity refname=REGP_2
192 ## OBJ PREDS ##
193 [conv2d] input=REG_0 channel=1 weight_order=wa act=sigmoid refname=OBJP_0
194 [conv2d] input=REG_1 channel=1 weight_order=wa act=sigmoid refname=OBJP_1
195 [conv2d] input=REG_2 channel=1 weight_order=wa act=sigmoid refname=OBJP_2
196
197 [concat] input=REGP_0,OBJP_0,CLSP_0 axis=1 [reshape] shape=$BATCH_SIZE,$CLASS_COUNT+5,-1
```

Layer-by-layer .cfg authoring with immediate compatibility validation against the original model.

# Completed the full inference pipeline after model output

Optimization did not stop at the network. I also implemented model-specific output decoding logic.

```
__global__ void kernel_yolox_postproc(float* output, float* input, float gain, int tcount)
{
    int pos = threadIdx.x + blockIdx.x * blockDim.x;
    if (pos >= tcount) return;

    int base = pos * 6;
    float x1 = input[base + 0];
    float y1 = input[base + 1];
    float x2 = input[base + 2];
    float y2 = input[base + 3];

    x1 /= gain;
    y1 /= gain;
    x2 /= gain;
    y2 /= gain;

    output[base + 0] = (float)((int)x1);
    output[base + 1] = (float)((int)y1);
    output[base + 2] = (float)((int)x2);
    output[base + 3] = (float)((int)y2);

    output[base + 4] = input[base + 4];
    output[base + 5] = input[base + 5];
}

void cuda_yolox_postproc(float* output, float* input, float gain, int N, int bbox_count, cudaStream_t cudaStream)
{
    int tcount = N * bbox_count;
    int block = 64;
    int grid = (tcount - 1) / block + 1;

    kernel_yolox_postproc << grid, block, 0, cudaStream >> (output, input, gain, tcount);
}
```

Example: YOLOX CUDA post-processing kernel and host wrapper.

## Output tensor analysis

Debugged model outputs to understand tensor layout, score encoding, and task-specific decoding.

## Native modules + postproc layer

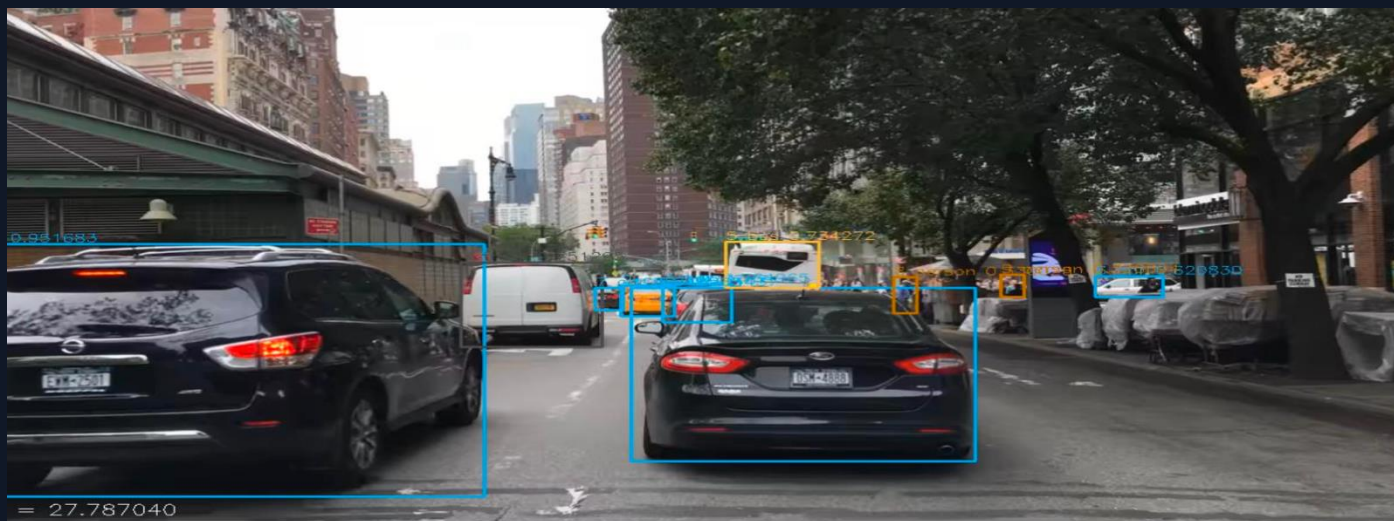
Implemented detection, segmentation, and classification postprocessing in C++ and CUDA.

## Engine-ready integration

Kept the optimized pipeline modular so it could be reused across model families.

# YOLOX-X: faster inference with lower GPU memory

The optimized engine delivered higher throughput and reduced GPU memory usage while matching the original model behavior.



FPS

**16-17 -> 38-39**

Approx. 2.3x faster

GPU MEMORY

**1800 -> 800MB**

Approx. 55.6% lower

Before vs. after

FPS

16-17

38-39

Memory

1800MB

800MB

Detection demo and runtime logs from the optimized YOLOX-X benchmark.

# What this internship demonstrates

This project is evidence of production-oriented AI engineering: not just using models, but making them efficient, portable, and measurable.

## Model-level debugging

Read and debugged open-source model code, tensor flows, and preprocessing logic.

## Systems implementation

Implemented C++/CUDA modules and custom layers for an optimized runtime.

## Verification discipline

Validated each layer against the baseline to preserve output correctness.

## Performance mindset

Benchmarked FPS and GPU memory to quantify engineering impact.